Architecture for CHESS MANIA!

Mark Starzynski
André Vieira Ribeiro

FS21

1. Introduction

This documentation outlines the general software architecture of the game Chess Mania. It uses a number of different architectural views to depict different aspects of the overall software system.

It will focus on outlining important aspects and processes of the game to provide the reader with a better understanding of the architectural structure. The accompanying views will depict the general client system, the general server system, the GUI, the game/lobby system as an important part of creating a game and the gamelogic system specifically. It should be noted that the gamelogic system specifically is nowhere near optimal and could use a lot of improvement.

2. Architectural Goals and Constraints

First and foremost, the development of this game was restricted to the use of a completely text-based networking protocol. The goal was to develop a 4-player multiplayer game in Java, which makes use of the programming language's already implemented socket programming for the connection between server and client instance.

The game needs to be implemented as a client-server system. The client system should be executed via one JAR file. The server system should be executed with the same JAR file and must be able to run separately from the client, but on the same PC system if needed.

The server instance also should be able to handle multiple clients without needing any further action besides the initial start.

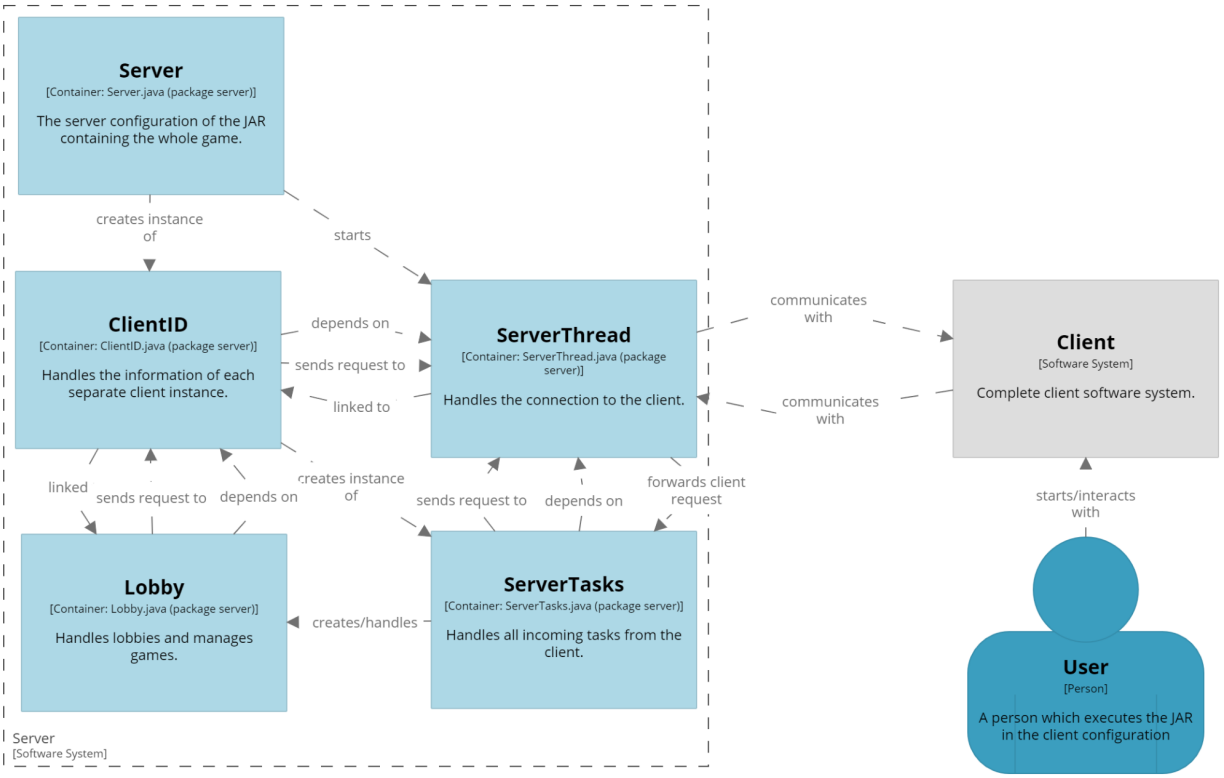
Clients should be able to connect to the server independently and access all functionality via a GUI, including playing the game with other clients.

The implementation of lobbies is mandatory and used in the architecture design as an important anchor point for the process of playing the game and maintaining its status.

The following sections will, step by step, elaborate on the decisions made regarding software architecture, focusing on a specific important aspect of the game.

3. Server Software System

The server software system is the complete implementation of the JAR file executed in the server configuration. The model view shown next displays the most important relations between the building blocks of this system and will be elaborated in the next paragraphs.



The server system handles all incoming client requests from clients connected to this server instance, as well as all outgoing server requests to all clients. The tasks it needs to fulfill are:

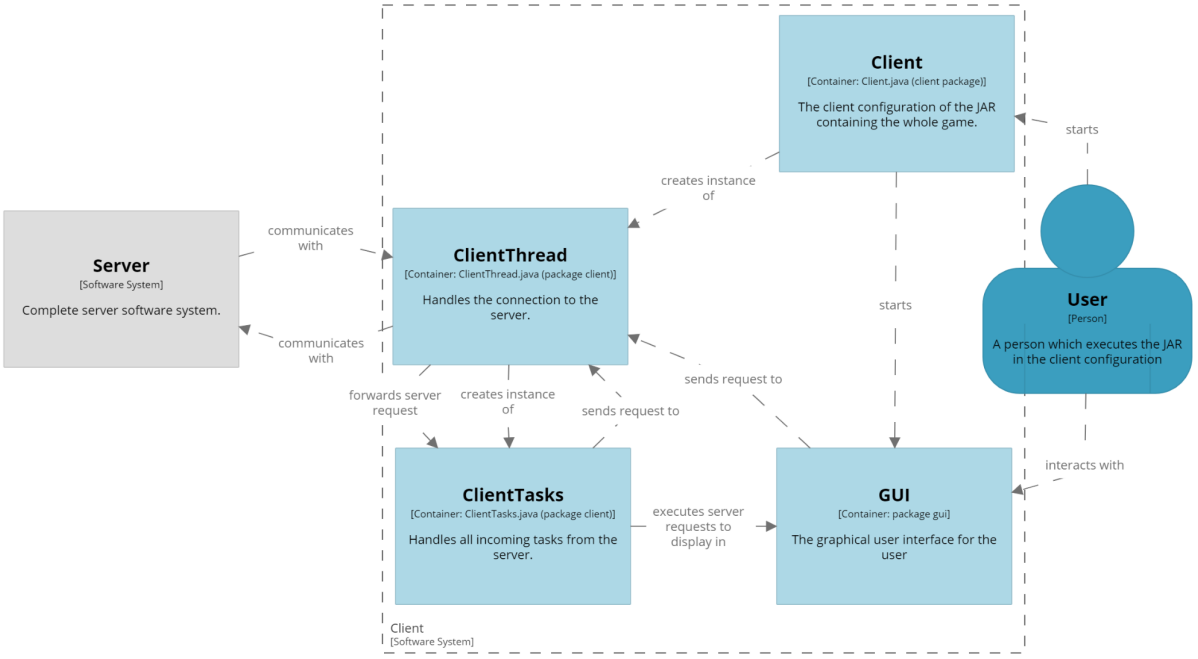
Login, logout, creating and maintaining of clients/lobbies/games/gamelists information, storing of a highscores list, handling of client requests (i.e. various independent chats, gameplay, name change), handling of the game logic and confirming a valid/invalid move, sending out task requests for all or single independent clients, maintaining connection to clients (pinging).

The design of the system is set out to run a server thread for each client separately. Accompanying each server thread is a ServerTasks instance, which handles incoming client requests by the specific user linked to the server thread.

Since one server usually processes all clients, the communication between the server and a specific client is linked to ClientIDs, unique identifiers for clients containing all data needed. Each ClientID, and hence by extension each client, has its own server thread, allowing the system to fulfill the plethora of tasks demanded.

4. Client Software System

The client software system is the complete implementation of the JAR file executed in the client configuration. The model view shown next displays the most important relations between the building blocks of this system and will be elaborated in the next paragraphs.



The client system handles incoming server requests as well as outgoing client requests to the server. The tasks it needs to fulfill are:

Making all necessary operations available to the user via GUI, playing sounds and animations, connecting to server, handling of server requests (i.e. receiving and using data), displaying the received information, maintaining connection to server (pinging).

Analogous to the server system, this design is set out to run a client thread for each client separately and have both server and client thread uniquely connected via socketing. Accompanying each client thread is a ClientTasks instance, which handles incoming server requests.

The view above shows the graphical interface, the GUI, in a simplified model and will be discussed in the next chapter.

5. GUI System

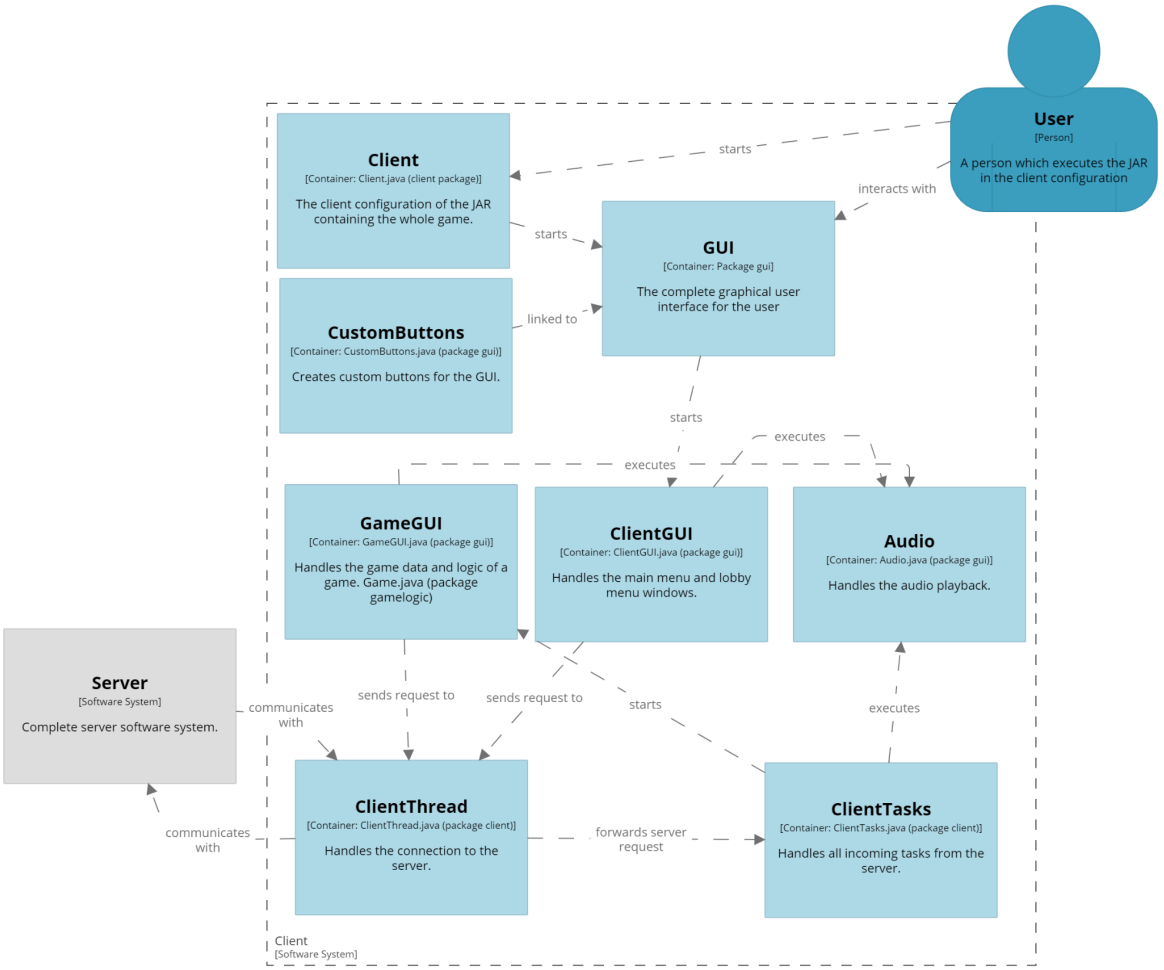
The GUI consists mainly of two separate stages depicted in the view model below as ClientGUI and GameGUI. Ideally, those two systems would be merged together more seamlessly, but time constraints and scarce human resource availability has forced the depicted end result.

The GUI starts initially with the ClientGUI, which displays all stages except the game stage to the user and lets them interact with it. Part of it is the intro, main menu, all sub menus and the separate lobby menu.

Through interaction with this stage, the user can create or join a lobby and then send a request to the server to start a game. If the server deems the request as valid and all conditions are met, the system will relay to the ClientTasks instance that a game can be started. From there the GameGUI will be started, which in turn allows the player to play the game.

This example of a use case can be applied to almost all tasks where the client needs to interact with the server. Another scenario depicted in the view model is the playback of a sound in the GUI, triggered by the server.

It should be noticed that the model “GUI” in the graphic exists solely in pretence to make the visualization and its relations more clear.



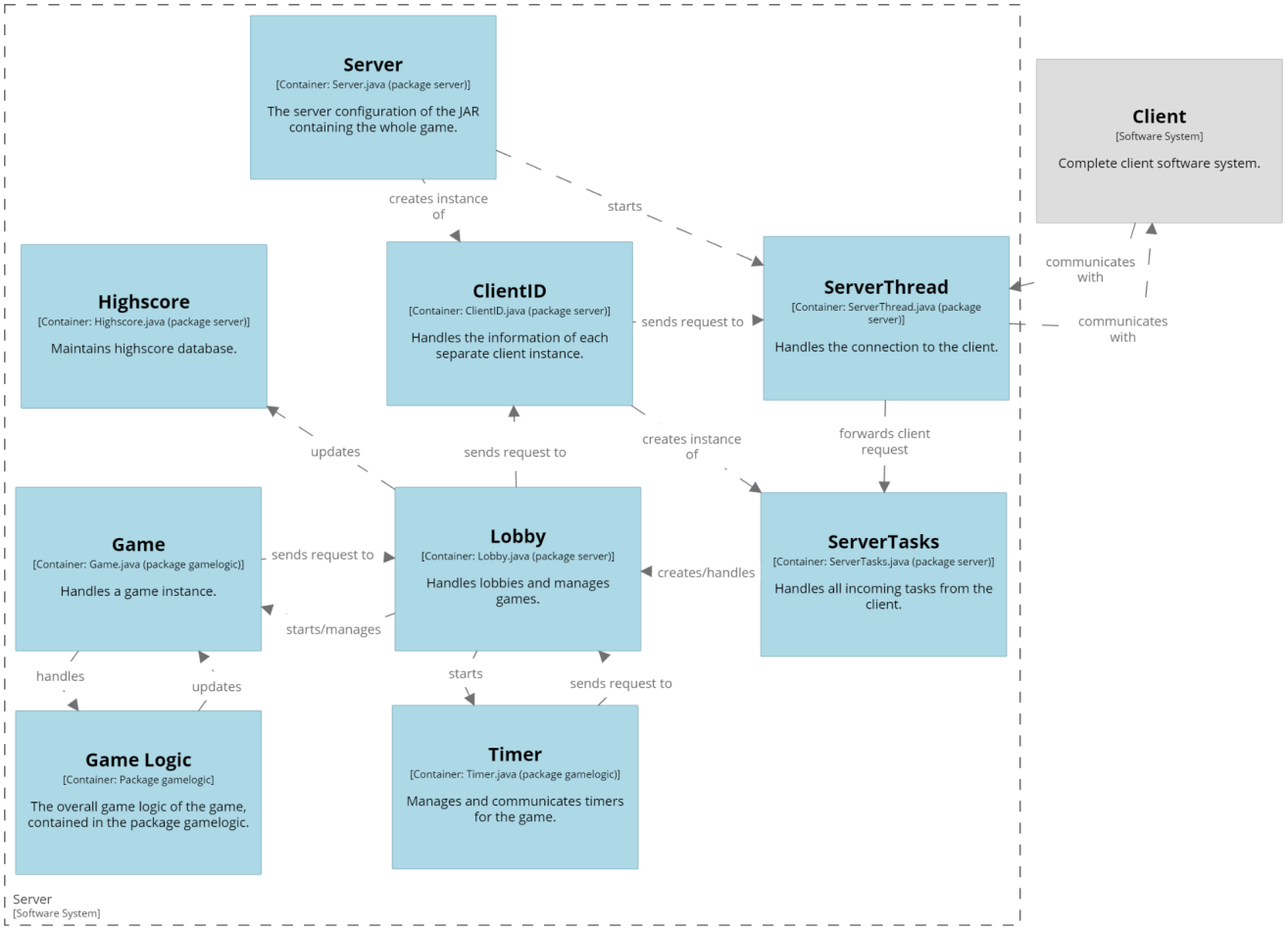
6. Game/Lobby System

The game/lobby system builds on the already mentioned server software system, but cuts some less important relations between the components in order to make room for the overall process of how a game is created and how lobby creation works in this architecture.
If a user requests to create a lobby, this gets relayed via server tasks to the lobby class, which stores the ClientIDs of relevant users. If a user requests to start a game, the task will be further relayed to the game class and so on.

Nevertheless, the lobby is the central anchor point of the game where player and game information is stored, timers for the game are executed and tracked and requests to clients relayed to the ClientID class, where it's possible to send the via linked server thread to the client.

After the game has ended the score gets updated via lobby as well.

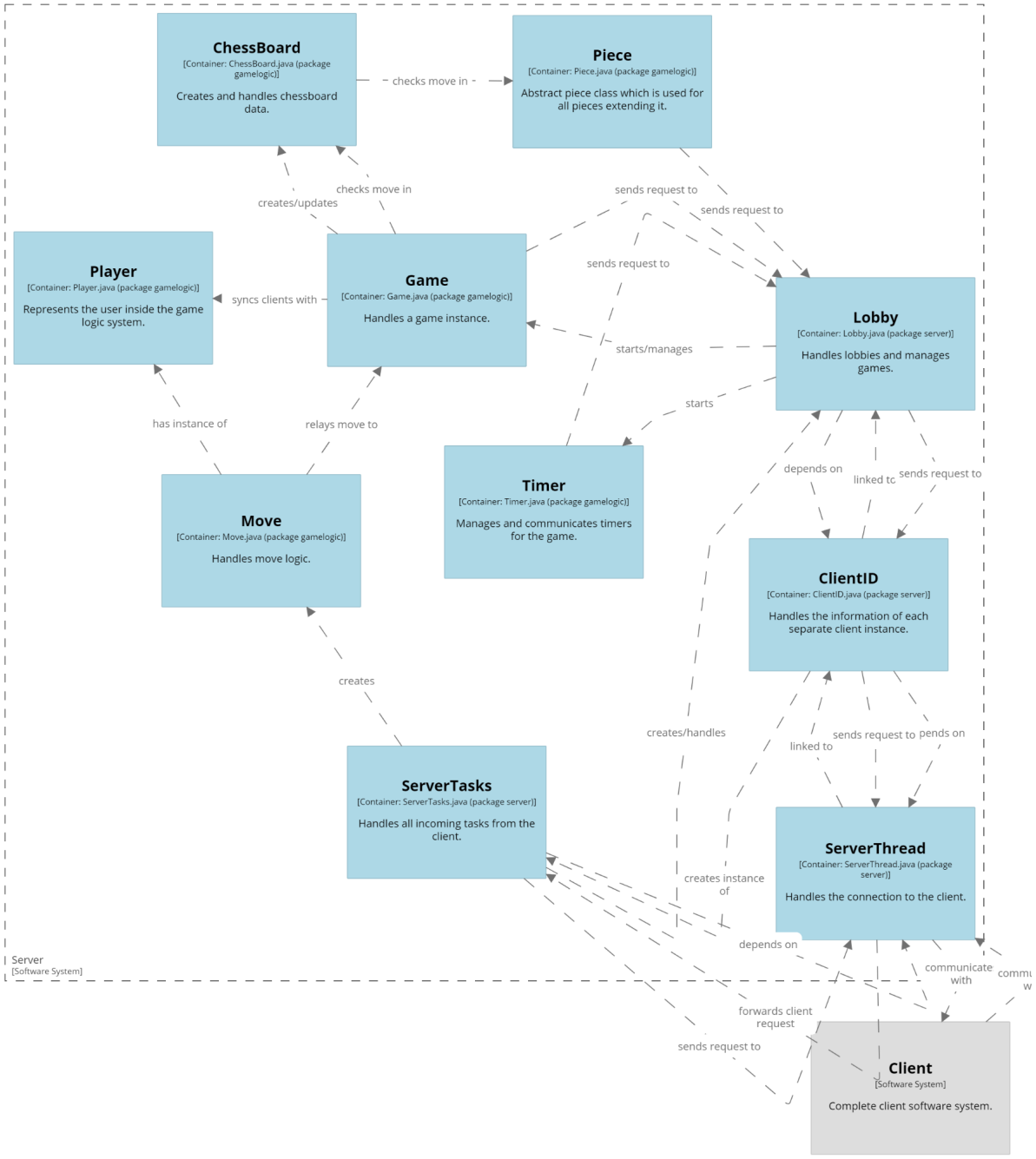
The game logic component is another placeholder for a more complex view model and will be expanded upon in the next chapter.



7. Game Logic System

The game logic system of the game is probably the most rough implementation of components in a sense that it isn't integrated very efficiently. The player class is technically not necessary at all, but implementations of different aspects of the game from different developers at times and certainly a lack of communication has left its marks on this part of the software architecture.

The careful reader is advised to not focus too much on the densely populated relations between the components on the lower right, those have been previously covered and are displayed for completion mainly.



The view model depicts the complicated process of a move request by a user playing a game. First the server gets the move order and relays it via server thread directly to server tasks, where the move gets created.

At game start the Game class synced the player class to the clientIDs already. This process could be removed for version 2.0 of the game by rewriting the code to only need one user identification class.

The created move, containing the player information as well, gets relayed to the game instance, which can be accessed via lobby. This instance then checks with the chessboard and piece classes whether the move is valid, where, depending on the quality of the move, the information gets passed on back to the lobby containing the game.

8. Final Remarks

This architecture documentation displayed, based on various use cases and generalizations, how the main components of the game work together to accomplish the set goals.

Some parts certainly could be improved for readability and performance, but the set time frame won’t allow this in the near future. Overall the designed software architecture accomplishes all target goals. It supports multiple clients and has a headless server. It can store highscores persistently and manages all sensible data server-sided. All known issues have been fixed or handled and the systems run stable for the most part. The game performance is very good for computers of this decade.